

Método Aprender

Relatório Técnico de Arquitetura e Análise do Repositório



Repositório: **ctorres4564/metodo-aprender**

Gerado automaticamente via análise do código-fonte

Visão Geral do Projeto

Método Aprender é uma plataforma web em português para estudo que combina repetição espaçada, recuperação ativa e geração de conteúdo assistida por IA com métodos pedagógicos (especialmente a Técnica Feynman). O público-alvo são estudantes autônomos que querem estudar qualquer assunto usando técnicas comprovadas pela ciência cognitiva.

O projeto é hospedado na **Vercel** (funções serverless) com **Firebase** como backend-as-a-service (Auth, Firestore, Storage). Não há etapa de build nem bundler — o frontend é HTML/CSS/JavaScript puro (`package.json` `"type": "module"` , linha 4).

Stack de Tecnologia

Camada	Tecnologia	Observações
Frontend	HTML/CSS/ES modules puro	Sem framework, sem bundler
Hospedagem	Vercel	Frontend estático + API serverless
Autenticação	Firebase Authentication	E-mail/senha e Google
Banco de dados	Firestore	Regras aplicadas, particionado por UID
Armazenamento	Firebase Storage	PDFs, uploads de usuários
IA	OpenRouter (GPT-4o-mini padrão)	<code>api/_lib/openrouter.js</code>
Pagamentos	Stripe	Assinaturas, checkout, portal
E-mail	Resend	Lembretes diários via Vercel Cron
Rastreamento de erros	Sentry	Apenas server-side
PWA	Service worker + manifest.json	Estratégia network-first
Testes unitários	Vitest	<code>npm test</code>
Testes de integração	Firebase Emulators + Vitest	<code>npm run test:emulator</code>
Testes E2E	Playwright	<code>npm run test:e2e</code>

Estrutura do Repositório

```
metodo-aprender/  
├─ *.html           # 10 páginas frontend (sem roteador)  
├─ assets/          # JS e CSS compartilhados do frontend  
├─ api/              # Funções serverless da Vercel
```

```

|   └─ _lib/                # Helpers compartilhados server-side
├─ content/                 # Módulos de estudo curados (JSON)
├─ test/                    # Todos os testes
|   └─ unit/                # Testes unitários puros (Vitest + JSDOM)
|   └─ emulator/            # Testes de integração com emulador Firebase
|   └─ e2e/                 # Testes de navegador com Playwright
|   └─ benchmark/           # Benchmarks de qualidade de avaliação por IA
├─ docs/                    # Notas de arquitetura e segurança
├─ scripts/                 # Scripts utilitários
├─ firestore.rules.txt      # Regras de segurança do Firestore
├─ storage.rules.txt        # Regras de segurança do Storage
├─ firebase.json            # Configuração do projeto Firebase
├─ manifest.json            # Manifesto PWA
└─ sw.js                    # Service worker

```

Arquitetura do Frontend

Todas as páginas são arquivos HTML independentes. O comportamento compartilhado é carregado via tags `<script src="assets/...">`. Arquivos principais:

assets/engine.js (2.387 linhas)

O motor central de aprendizado. Contém:

- **Implementação do FSRS 4.5** — algoritmo de agendamento por repetição espaçada. O estado é gerenciado por conceito por usuário (objeto `STATE` com `cards`, `streak`, `xp`, `quiz` etc.)
- **Gamificação** — XP, 10 níveis, 11 definições de badge (linhas 76–94)
- **5 modos de estudo**: Aprender, Revisar, Explicar (Feynman), Quiz, Progresso
- **Máquina de estados da avaliação Feynman** — `EXPLAIN_ATTEMPT_STATUSES` e `EXPLAIN_PEDAGOGICAL_DECISIONS_FROM_API` (linhas 131–141): tentativas são salvas como `pending_evaluation` antes de a API ser chamada, de forma que o texto nunca se perde mesmo que a IA falhe
- **Rastreamento de decisão pedagógica** — `COMPREHENSION_STATUSES`, `CALIBRATION_STATUSES`, `RETRIEVAL_SOURCES` (linhas 142–145)

assets/storage.js (138 linhas)

Classe `StorageAdapter`: grava progresso no `localStorage` imediatamente, sincroniza com o Firestore com debounce. Oferece fallback em caso de indisponibilidade de rede.

assets/api-client.js

Wrapper fino sobre `fetch` que anexa um bearer token do Firebase Auth a cada requisição.

assets/module-drafts.js

Gerencia rascunhos com salvamento automático no `localStorage`, com chave composta por `uid + draftId`. Rascunhos expiram após 30 dias e são removidos ao salvar no Firestore.

assets/onboarding-wizard.js

Wizard para primeira visita: detecta novos usuários (sem módulos), substitui botões de ação por dois caminhos guiados — "Tenho um PDF ou texto" (`criar-modulo.html#import-ai`) e "Sei o que quero estudar" (editor manual).

assets/ocr.js

OCR client-side via Tesseract.js para PDFs digitalizados.

API / Funções Serverless

Todas em `api/`, cada uma sendo um único arquivo de função serverless da Vercel.

Endpoints com IA (exigem token Firebase válido + cota)

Arquivo	Finalidade	Balde de cota
<code>api/avaliar-explicacao.js</code>	Avaliação de explicação pelo método Feynman	<code>explain</code>
<code>api/gerar-modulo.js</code>	Gerar conceitos a partir de texto/PDF	<code>generate</code>
<code>api/regenerar-conceito.js</code>	Regenerar um único cartão de conceito	<code>generate</code>
<code>api/localizar-secao.js</code>	Encontrar uma seção de livro por descrição	<code>generate</code>
<code>api/gerar-analogia.js</code>	Gerar uma analogia para um conceito	<code>explain</code>

Endpoints de biblioteca / administração

Arquivo	Finalidade
<code>api/material.js</code>	CRUD consolidado da biblioteca (create, updateStatus, registerModule, recordRead, delete)
<code>api/account.js</code>	Exclusão de conta: cancela Stripe, exclui dados do Firestore, exclui usuário do Auth
<code>api/enviar-lembretes.js</code>	Lembretes por e-mail diários (chamado pelo Vercel Cron às 12:00 UTC)

Endpoints de pagamento

Arquivo	Finalidade
<code>api/criar-checkout.js</code>	Criar sessão de Checkout no Stripe
<code>api/stripe-webhook.js</code>	Sincronizar eventos de assinatura do Stripe com <code>users/{uid}.plan</code> no Firestore
<code>api/stripe-portal.js</code>	Abrir o Portal de Cobrança do Stripe

Biblioteca Server-Side Compartilhada (api_lib/)

usage.js — Controle de Cota

A camada de controle de cota. Decisões de design principais:

- **Dois baldes de cota:** `explain` (300/3.000 por mês) e `generate` (60/600), além de `constructed_eval` (300/3.000). Separados porque o custo por chamada é drasticamente diferente (linhas 19–37).
- **Consumo transacional de cota:** usa transações do Firestore com retry exponencial de até 4 tentativas para erros `ABORTED` (linhas 98–162). Retorna `transient_error` em vez de lançar exceção.
- `refundUsage()` (linhas 178–196): se a chamada à IA falhar após o consumo da cota, o contador é decrementado. Evita cobrar o usuário por requisições que não entregaram resultado.
- `requireUsageQuota()` (linhas 198–240): helper de chamada única usado pelos 5 endpoints de IA; trata 401/403/429/503.
- **Barreira de verificação de e-mail:** e-mails não verificados são bloqueados do consumo de IA para evitar abuso via contas descartáveis (linhas 87–90).

openrouter.js — Chamador Centralizado de IA

- Timeout de 50 segundos via `AbortController` (linha 25), mantido ativo durante toda a leitura do corpo da resposta para prevenir ataques de "slow-drip".
- Cadeia de modelos de fallback: `[primário, gemini-flash-1.5, llama-3-8b:free]`.
- Códigos de erro tipados: `timeout | network | http | parse`.
- `statusForOpenRouterError()` mapeia códigos para status HTTP (504 para timeout, 502 para os demais).

Outros módulos

- `explanationEvaluation.js` : prompt de avaliação Feynman e normalização da resposta. Extraído para possibilitar testes sem rede.
- `constructedEvaluation.js` : lógica de avaliação por IA da resposta construída (texto livre).
- `sanitize.js` : sanitização de entrada (`cleanStr`). O frontend usa `escapeHtml()` / `textContent` para prevenção de XSS.
- `firebaseAdmin.js` , `stripe.js` , `sentry.js` : singletons inicializados de forma lazy.

Modelo de Dados (Firestore)

Coleção / Caminho	Conteúdo	Acesso
<code>users/{uid}</code>	Perfil, plano, remindersEnabled	Somente o dono; <code>plan</code> é escrita exclusiva do servidor
<code>progress/{uid}_ _{storageKey}</code>	Estado FSRS, XP, badges por módulo	Somente o dono (regex de correspondência)
<code>modules/{moduleId}</code>	Módulos criados pelo usuário	Somente o dono; <code>ownerId</code> imutável
<code>ai_usage/{uid}_{AAAA-MM}</code>	Contadores mensais de geração	Escrita pelo servidor; leitura pelo dono
<code>ai_usage/{uid}_{AAAA-MM}_ _explain</code>	Contadores mensais de explicação	Escrita pelo servidor; leitura pelo dono
<code>materials/{materialId}</code>	Metadados de livro/PDF	Escrita pelo servidor; leitura pelo dono
<code>materials/{materialId}/ pages/{n}</code>	Texto extraído por página	Leitura/escrita pelo dono
Subcoleções: highlights, notes	Anotações no leitor	Somente o dono

As regras do Firestore aplicam todos os padrões de acesso (`firestore.rules.txt`). Destaque: `ownerId` é imutável em atualizações de módulo (SEC-08, linha 29 das regras) para prevenir ataques de injeção de conteúdo.

Conteúdo Curado (content/)

Três módulos oficiais, cada um como um arquivo JSON versionado:

1. [tecnica-feynman.json](#) — 10 conceitos sobre a Técnica Feynman
2. [repeticao-espacada-e-memoria.json](#) — ciência da repetição espaçada e memória
3. [metacognicao-aprender-a-aprender.json](#) — metacognição

Formato de módulo:

```
{
  "config": { "storageKey": "estudo_tecnica_feynman_v1", ... },
  "concepts": [
    { "id": "...", "tag": "...", "title": "...", "text": "...",
      "q": "...", "options": [...], "correct": 0 }
  ]
}
```

`storageKey` e os `id`s dos conceitos são identificadores estáveis — alterá-los apagaria silenciosamente o progresso existente dos usuários.

Estratégia de Testes

Três suítes executadas no CI ([.github/workflows/tests.yml](#)):

1. `npm test` (**Vitest**) — testes unitários para: motor FSRS, parsing, sanitização, helpers de uso, wizard de onboarding, rascunhos de módulos, Sentry, OpenRouter, identidade de conceitos, regressões de segurança/XSS ([test/security-hardening.test.js](#))
2. `npm run test:emulator` — testes de integração contra emuladores reais do Firebase: regras do Firestore ([test/emulator/firestore.rules.test.js](#)), regras do Storage, segurança de concorrência na cota ([test/emulator/usage.concurrency.test.js](#)), processamento de webhook do Stripe, exclusão de conta
3. `npm run test:e2e` (**Playwright**) — fluxos completos no navegador: autenticação, criação de módulo, leitor de PDF, explicação Feynman, resposta construída

Uma quarta suíte em [test/benchmark/](#) mede a acurácia da avaliação por IA contra datasets rotulados — não executada no CI, usada para acompanhamento da qualidade dos modelos.

Postura de Segurança

Medidas principais (documentadas em [docs/security-known-risks.md](#)):

- **Verificação de token Firebase** em todos os endpoints via `verifyUserFromRequest()`
- **Regras do Firestore** aplicam propriedade baseada em UID em todas as coleções
- **Campo `plan`** é escrita exclusiva do servidor; as regras do cliente o bloqueiam explicitamente (`firestore.rules.txt` linhas 40–44)
- **Validação de assinatura do webhook do Stripe** (`api/stripe-webhook.js`)
- **Prevenção de XSS:** HTML dinâmico usa `escapeHtml()` ou `textContent` ; cabeçalhos CSP enviados pela Vercel
- **Lacuna conhecida:** a CSP ainda inclui `'unsafe-inline'` devido a scripts inline (documentado no README e em [docs/security-known-risks.md](#))
- **Exclusão de conta** (`api/account.js`) cancela a assinatura do Stripe, exclui todos os dados do Firestore e então exclui o usuário do Auth — nessa ordem, para evitar cobrança órfã

Decisões Arquiteturais Principais

1. **Sem framework, sem bundler:** reduz complexidade e sobrecarga de tooling de build. A contrapartida é o gerenciamento manual de dependências via tags `<script>` e a ausência de tree-shaking.
2. **`engine.js` é agnóstico ao conteúdo:** `initApp(config, concepts)` é o único ponto de entrada, permitindo que qualquer módulo JSON seja executado sem alterações no código.
3. **Barreira de revisão humana para módulos gerados por IA:** conteúdo gerado é salvo apenas como rascunho no `localStorage` . A escrita no Firestore só acontece quando o usuário clica em "Salvar módulo". Previne que conteúdo acidental da IA seja persistido.
4. **Padrão de quota-antes-da-chamada:** a cota é consumida transacionalmente *antes* da chamada à IA, com `refundUsage()` em todos os caminhos de falha. É mais seguro do que a contabilização pós-chamada sob acesso concorrente.
5. **Garantia de estabilidade do `storageKey` :** alterar o `storageKey` de um módulo ou os IDs de conceitos silenciosamente tornaria órfão o progresso existente dos usuários — documentado explicitamente no README.